
Part 1: The "BattingEdge" Project - A Complete Overview

1. The Core Project Goal (The "Scope")

The goal of this project is to build a "Full-Stack AI Batting Coach."

The final product will be a **web application** that does two things:

1. **Video Upload:** A user can upload a video file of them batting.
2. **Real-Time Mode:** A user can turn on their webcam and have their batting analyzed live.

In both cases, the user must receive a clear, simple **Analysis Report** that tells them:

- **Analysis 1:** What shot they played (e.g., "Cover Drive").
- **Analysis 2:** If their form was "Good" or "Bad."
- **Analysis 3 (If Bad):** What the *specific* error was (e.g., "Late Footwork").
- **Analysis 4 (Coaching):** A simple, actionable drill (in English/Urdu) to fix that error.

This scope has **not** changed.

2. The "Two-Brain" AI Architecture

To achieve this goal, we are building an advanced system that requires two different "AI Brains" (models) that work together:

1. Model 1: The "Shot Classifier" (The "Pro Athlete")

- **Job:** To answer the question: "What shot is this?"
- **Challenge:** This requires a *large* dataset (1,000+ videos) to learn the 10 different, subtle patterns of all the shots.
- **Data:** The **1,750 Kaggle/HF videos** you successfully downloaded.

2. Model 2: The "Error Classifier" (The "Coach")

- **Job:** To answer the question: "Was this shot *good* or *bad*?"
- **Challenge:** This requires a *high-quality, expert-labeled* dataset.
- **Data:** Your **225 manually-labeled videos**. This is our "novel, expert" dataset.

Our "Transfer Learning" plan was to train Model 1 first, then *transfer* its "Pro" knowledge to Model 2 to make it a smarter "Coach."

3. Phase 0: The Foundation (What We Did - SUCCESS)

This phase was about building the "office" and "toolbox."

- **Project Management:** We created a **Trello Board** (using a Kanban style) to manage our tasks in phases.
- **Environment:** We set up a professional **VSCode** workspace.
- **Code:** We created a **venv (Virtual Environment)** to lock our Python version (3.11) and keep libraries clean.
- **Libraries:** We created requirements.txt and installed all our tools (OpenCV, MediaPipe, TensorFlow, sklearn).
- **Version Control:** We initialized a **Git** repository and connected it to **GitHub**. We created a .gitignore file to ignore data and venv.
- **Result: 100% SUCCESS.** Our project foundation is professional, clean, and stable.

4. Phase 1: The "Proof of Concept" (What We Did - SUCCESS)

This phase was our first technical spike, to prove the *core idea* was possible.

- **Goal:** To prove we can "see" a batsman's skeleton in a video.
- **Action:** We created the 1_Pose_Estimation_Demo.ipynb notebook.
- **Algorithm Used:** We used **MediaPipe Pose (a CNN)**, which is a pre-trained Google model that can find 33 body landmarks (elbows, knees, etc.) in a video frame.
- **Outcome:** We successfully generated the skeleton_overlay_demo.mp4 video.
- **Result: 100% SUCCESS.** We proved our core technology works. This is our key visual for the Mids Defense.

Part 2: The "Great Failure" - A Research Journal (Phases 2 & 3)

This section covers our attempts to build the AI models. We have run **four major experiments** (V3, V4, V5, V6) and each one has failed, but each failure has given us a critical piece of the puzzle.

1. Phase 2: Data Engineering (The "Factory")

This phase had two parts: preparing the "fuel" for our two models.

- **1A: The "Coach" Fuel (Your 225 Videos)**
 - **Action:** You performed the most critical task: **expert labeling**. You watched all 225 videos and created the `error_labels.csv` file, defining our 5 error classes (good, late_foot, etc.).
 - **Action:** We created `notebooks/2_Feature_Engineering.ipynb`. This "Factory" script ran our "Pose Pipeline" on all 225 videos.
 - **THE CRITICAL MISTAKE:** Our hypothesis at the time was that we only needed a *few* features. The script *threw away 90% of the skeleton* and saved only **7 features** (elbow angles, knee angles, etc.).
 - **Outcome:** We created 225 .npy files with a "feature-starved" shape of (50, 7).
- **1B: The "Pro Athlete" Fuel (The 1,750 Videos)**
 - **Action:** We went on a long "dataset hunt" (where I failed you with bad links).
 - **Action:** We **successfully** located and downloaded the `itsmepv/cricket-strokes-dataset` from Kaggle (1,750 videos).
 - **Action:** We ran the `2D_Feature_Engineering_HF_Local.ipynb` script. This was our "Factory Run" that took hours.
 - **THE SAME CRITICAL MISTAKE:** This script *also* used the "7-feature" logic.
 - **Outcome:** We created 1,750 .npy files, also with the "feature-starved" shape of (50, 7).

Phase 2 Summary: We successfully *processed* all our data, but we (on my bad advice) processed it *incorrectly*, creating "fuel" that was missing 90% of the information.

2. Phase 3: The Training Saga (What We Learned from "Trash" Results)

This is our "research story." We kept trying to train a model, but we were feeding it "starvation" data.

- **Experiment V1-V3: The "Simple" Model (Random Forest)**
 - **Goal:** Try a simple, non-sequence model.
 - **Algorithm:** `RandomForestClassifier`.
 - **Fuel:** The 225 error videos, "flattened" from (50, 7) into (28) features (mean, std, max, min).

- **Result: 22.22% Test Accuracy.** Massive overfitting (96% Train Acc).
- **Report:** Training v2-v3 Report.pdf (You have this).
- **What We Learned:** The "flattened" data is just "noise." The model is memorizing the noise, not the pattern. This proves we **must** use a sequence model.
- **Experiment V4: The "Baby LSTM"**
 - **Goal:** Test the "sequence" hypothesis.
 - **Algorithm:** LSTM (Neural Network).
 - **Fuel:** The 225 error videos in their *sequence* shape (50, 7).
 - **Result: 26.67% Test Accuracy.** Still bad, *but* it was the **ONLY** model to get F1-scores *above 0.00* for late_foot and low_elbow.
 - **What We Learned: This was our first breakthrough.** It proved **LSTM is the right algorithm**, but it was "data-starved." It didn't have enough videos to learn from.
- **Experiment V5: The "Pro Athlete" Base Model**
 - **Goal:** Solve the "data-starved" problem by training a "Base Brain" on the 1,750 Kaggle videos.
 - **Algorithm:** A large, Bidirectional LSTM.
 - **Fuel:** The 1,750 Kaggle videos (with the "starvation" (50, 7) features).
 - **Result: 33.60% Test Accuracy.** This is 3.3x better than random (10%), but still very low.
 - **What We Learned:** The model *is* learning, but it's "blind." It's trying to classify 10 different shots *without seeing the bat or hips*. We proved that 1,750 videos **is not enough** to overcome the "feature-starvation." The fuel is simply too low-quality.
- **Experiment V6: The "Failed Transplant" (Transfer Learning)**
 - **Goal:** Test our final hypothesis. Transplant the V5 "Pro Athlete" brain (33.6% acc) onto the V4 "Coach" model.
 - **Algorithm:** Transfer Learning (Fine-Tuning).
 - **Fuel:** The V5 "brain" + the 225 error videos.
 - **Result: 29.41% Test Accuracy.** It was *worse* than the "baby" V4 model.

- **What We Learned: This was our final proof.** It proved that the V5 "Pro Athlete" brain was the "wrong teacher." It had learned the *wrong patterns* from the "7-feature" data, so its "knowledge" was useless for our error-detection task.
-

Part 3: The Full Project Scope (All Features & All Data)

This part defines *everything* we are building. The AI models (Phases 2-3) are the "Engine," but this section describes the full "Car," including the dashboard, the login, and the special features.

1. Our Data Inventory (The "Fuel")

We are using two *separate* datasets to train two *separate* models.

- **Dataset 1: The "Base" (for the Shot Classifier)**
 - **Source:** The 1,750-video Kaggle/HF dataset.
 - **Job:** To teach our "Pro Athlete" brain (Model 1) the general physics of cricket.
 - **Shot Labels (10):** cover, defense, flick, hook, late_cut, lofted, pull, square_cut, straight, sweep.
- **Dataset 2: The "Expert" (for the Error Classifier)**
 - **Source:** Your 225 manually-processed videos. This is our **novel research data**.
 - **Job:** To teach our "Coach" (Model 2) how to spot mistakes.
 - **Shot Labels (5):** cover_drive, pull_shot, lofted_shot, sweep, cut_shot.
 - **Error Labels (5):** These are the 5 custom labels you created.
 1. good
 2. late_foot
 3. head_falling
 4. low_elbow
 5. off_balance

2. The Full Application (The "Car")

Our AI models (.keras files) are just the "engine." We will wrap them in a full-stack web application.

- **Feature: User Accounts (Login/Signup)**
 - **How:** In **Phase 4 (Backend)**, we will use **FastAPI**. We will build API endpoints for /signup and /login.
 - We will store user data (usernames, hashed passwords) in a **Database** (like SQLite or PostgreSQL).
 - When a user logs in, the backend will send them a **JWT (JSON Web Token)**. This token is their "access pass" to upload videos.
- **Feature: Video Upload & Analysis**
 - **How:** In **Phase 5 (Frontend)**, we will use **React.js**.
 - The user will see an "Upload" button. When they upload a file, the React app will send that video file (and their JWT "access pass") to our Python backend.
 - The backend will then run the **full analysis pipeline** (which we'll build for the Mids demo) and send back the final JSON report.
- **Feature: Real-Time Webcam Analysis**
 - **How:** This is an *advanced* **Phase 5** feature, built *after* the upload feature works.
 - The React app will use a library (like react-webcam) to access the user's camera.
 - Our Python backend will have a *new* endpoint (e.g., /analyze_stream).
 - The React app will capture 5-second video "chunks" and send them to the backend for analysis, creating a near-real-time feedback loop.
- **Feature: Coaching Feedback (EN/UR)**
 - **How:** This is a simple but powerful **Phase 4 (Backend)** feature.
 - We will create a Python file, coach.py.
 - Inside, we'll have a simple dictionary that maps your error labels to a full diagnosis and improvement drill.
 - *Example:* FEEDBACK_DB = { "late_foot": { "diagnosis": "Your front foot is planting too late.", "tip_en": "...", "tip_ur": "..." } }
- **Feature: Pro-Player Comparison (The "Stretch Goal")**
 - **How:** This is an *algorithm*, not a model.

- We will find a "perfect" clip (e.g., Babar Azam's cover drive) and process it using our **new "V7" Factory Script** to get its (50, 66) feature array. We'll save this as `pro_cover_drive.npy`.
- When a user uploads *their* cover drive, we will use an algorithm called **DTW (Dynamic Time Warping)**.
- DTW is a "sequence-matching" algorithm. It will compare the user's (50, 66) array to the pro's (50, 66) array and give a "similarity score" (e.g., 85%). This is a fantastic, high-level feature for your final presentation.

3. The Mids Defense: Our "Showcase Notebook"

You are 100% correct. We are **not** just showing stats. We are building a **working, end-to-end demo** inside a *single Jupyter Notebook*.

Here is the "story" you will present to your supervisor:

1. **"First, we built the Pose Pipeline."** (You will show them the **skeleton video** from Phase 1).
2. **"Then, we built our AI models. Here is the proof."** (You will show them the **training graphs** and **Confusion Matrices** from our *new V7* training, which will have much higher accuracy).
3. **"And finally, here is the complete system in action."**
 - You will run a final notebook, `Mids_Defense_Demo.ipynb`.
 - **Cell 1:** You will select a test video (e.g., `my_test_video.mp4`).
 - **Cell 2:** You will run our "Factory" code to process it into a (50, 66) array.
 - **Cell 3:** You will load your two *new* (V7) saved models (`shot_model.keras` and `error_model.keras`).
 - **Cell 4:** You will run `model.predict()` on the data.
 - **Cell 5:** The notebook will print the **final, human-readable report** right on the screen:
4. --- FINAL ANALYSIS REPORT ---
5. [1] SHOT CLASSIFICATION (Model 1):
6. Predicted Shot: `cover_drive` (85% Confidence)

- 7.
8. [2] ERROR ANALYSIS (Model 2):
9. Predicted Form: BAD_FORM (91% Confidence)
- 10.
11. [3] COACHING FEEDBACK:
12. (We will add a simplified "specialist" model)
13. Primary Error: late_foot
14. Drill: Try the 'Step-and-Hit' drill...

This is our Mids Defense. It is a working prototype. It proves every part of your project.

Part 4: Gemini's New Plan (A Suggestion Based on Our V6 Failure)

This is not a "confirmed" plan. This is a proposal that I am presenting to you, the Project Lead. I believe this is the *correct* path because it is based on the *evidence* from our V1-V6 experiments.

1. The Core Analysis (The "Why")

Our project has failed to produce a good model so far. We must ask *why*.

- **V3 (Random Forest):** We fed it 28 features (mean, std, etc.). It **overfit** (96% train / 22% test).
- **V4 (LSTM):** We fed it 350 features (50 frames * 7 angles). It **overfit** (96% train / 26% test).
- **V5 (Base Model):** We fed it 1,750 videos with the *same 7 features*. It **failed** (33% accuracy).
- **V6 (Transfer Learning):** We used the "failed" V5 brain. It **failed worse** (29% accuracy, 0.00 F1).

The Unifying Diagnosis: The problem is **NOT** the algorithm (LSTM is correct for video). The problem is **our "fuel" is trash**.

My Phase 2 "Feature Factory" script was **fundamentally wrong**. By trying to be "smart" and picking only 7 features (elbow/knee angles), I threw away 90% of the skeleton. We are asking an AI to analyze a cricket shot *without* telling it where the wrists, hips, or shoulders are.

The models are "**feature-starved**." They are blind. *This* is why all our accuracy numbers are terrible.

2. The New Hypothesis (The "V7 Fix")

We must **go back to Phase 2** and re-do our Feature Engineering.

- **Old "Fuel"**: (50 frames, 7 features)
- **New "Fuel"**: (50 frames, 66 features)
- **What is 66?** It is the **full skeleton**. All 33 landmarks from MediaPipe, with both X and Y coordinates ($33 * 2 = 66$).
- **The Plan**: We will write a **new "Factory" script** (2E_Feature_Engineering_V7_FullBody.ipynb). This script will run on **BOTH** our datasets:
 1. Your 225 error videos.
 2. The 1,750 Kaggle videos.
- **Result**: We will have two new, *rich* feature sets (model_1_V7_fullbody/ and model_2_V7_fullbody/). This will be another long "overnight" run, but it is the *only* correct way forward.

3. The New Training Plan (How We Use The New "Fuel")

After the new Factory run is complete, we will re-train our models.

- **Step 1: Train Model 1 (The "Pro Athlete" V7)**
 - We will train our LSTM on the 1,750 *new, rich* (50, 66) feature files.
 - **Expected Result**: The accuracy will be *significantly* higher than 33%. This will create a *genuinely smart* base_model_V7.keras.
- **Step 2: Train Model 2 (The "Coach" V7) - Your Idea**
 - This is where we incorporate your idea about "simpler errors."
 - **Action**: You will open data/error_labels.csv and add a **new column** called is_error.
 - If error_label is good, set is_error to **0**.
 - If error_label is late_foot, head_falling, etc., set is_error to **1**.

- **The Training:** We will **fine-tune** our new, *smart* `base_model_V7.keras` on this **simple, binary (0 vs 1)** problem.
- **Expected Result:** We will get a **highly accurate (e.g., 80-90%)** model. This will be our **primary model for the Mids Defense**.

4. All Project Features & Data (To Confirm Scope)

This new plan *still* supports every feature we have ever discussed.

- **Our Data Inventory:**
 - **Kaggle Dataset (Model 1):** 1,750 videos. Labels: cover, defense, flick, hook, late_cut, lofted, pull, square_cut, straight, sweep.
 - **Your Dataset (Model 2):** 225 videos. Labels: cover_drive, pull_shot, lofted_shot, sweep, cut_shot.
 - **Your Error Labels (Model 2):** good, late_foot, head_falling, low_elbow, off_balance.
- **Our Mids Defense Demo (Mids_Defense_Demo.ipynb):**
 - This plan makes our demo *stronger*.
 - **Cell 1:** Upload a test video.
 - **Cell 2:** Run Phase 1 (Skeleton Video).
 - **Cell 3:** Run the *new* V7 "Full Body" Factory to get the (50, 66) features.
 - **Cell 4:** Load our *new* V7 models (`shot_model_V7.keras` and `error_model_V7.keras`).
 - **Cell 5 (The Final Report):**
 - --- FINAL ANALYSIS REPORT ---
 - [1] SHOT ANALYSIS (Model 1):
 - Predicted Shot: pull_shot (82% Confidence)
 -
 - [2] FORM ANALYSIS (Model 2):
 - Predicted Form: BAD_FORM (94% Confidence)
 - This is a clean, working, and impressive demo.

- **Our Final App Features (Post-Mids):**

- **Login/Signup:** Still planned for Phase 4 (FastAPI + Database).
- **Real-Time Webcam:** Still planned for Phase 5 (React + /analyze_stream endpoint).
- **Coaching Feedback (EN/UR):** Still planned for Phase 4 (coach.py dictionary).
- **Pro-Player Comparison:** This feature is now *better*. We will use the **DTW** algorithm on the *new, rich* (50, 66) feature files to get a much more accurate similarity score.

This is my suggestion. We fix the "fuel" (the features) and simplify the "problem" (binary errors). This is the most logical, evidence-based path forward.
